

Introduction to jQuery

jQuery is a fast, small, and feature-rich JavaScript library that simplifies client-side scripting of HTML. It was created to make it easier to interact with the DOM, handle events, create animations, and perform AJAX operations with less code compared to plain JavaScript.

Feature of JQuery :

1. Simplifies JavaScript Code

- jQuery allows developers to write less code to achieve tasks that normally require many lines of JavaScript.
- Example: Selecting an element, adding a class, or hiding an element can be done in one line.

2. Cross-Browser Compatibility

- jQuery automatically handles differences between browsers, so developers don't need to write separate code for Chrome, Firefox, Safari, or Internet Explorer.

3. DOM Manipulation

- jQuery makes it easy to select HTML elements and manipulate their content, attributes, or style.

4. Event Handling

- jQuery provides simple methods to detect and respond to user interactions like clicks, key presses, or form submissions.

5. Effects and Animations

- jQuery includes built-in methods for showing/hiding elements, fading, sliding, and custom animations.

6. AJAX Support

- jQuery simplifies asynchronous requests to the server, allowing dynamic page updates without reloading.

7. Extensible

- Developers can write plugins to add new functionality, making jQuery highly extendable.

Add JQuery via CDN :

1. Find official JQuery CDN:

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

2. Paste the CDN inside your HTML file

```
<head>  
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>  
</head>
```

Example :

```
<!DOCTYPE html>  
<html>  
  
  <head>  
    <title>jQuery CDN Test</title>  
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>  
    <script>  
      $(document).ready(function () {  
        console.log("jQuery is successfully loaded!");  
      });  
    </script>  
  </head>  
  <body>  
  
  </body>  
</html>
```

Jquery Syntax :

`$(selector).action();`

Selector :

A jQuery selector is a rule or pattern that jQuery uses to locate specific elements in an HTML page.

It tells jQuery *which items* you want to work with, similar to how CSS selects elements.

Selector	Meaning / What It Selects
<code>\$("#title")</code>	Element with ID title
<code>\$(".box")</code>	All elements with class box
<code>\$("p")</code>	All <p> tags
<code>\$("div > p")</code>	<p> tags that are direct children of <div>
<code>\$("ul li:first")</code>	The first inside a list
<code>\$("input[type='text']")</code>	All text input fields
<code>\$("a[href*='google']")</code>	All links containing “google” in href
<code>\$("img[src\$='.png']")</code>	Images with .png extension
<code>\$("section .item")</code>	Items inside a section
<code>\$("button, a")</code>	Selects both <button> and <a>

Example :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>jQuery Selector Demo</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
  <h2>User Form</h2>
  <p>Fill out your details below:</p>
  <input type="text" class="inputBox" placeholder="Enter Name"><br><br>
  <input type="email" class="inputBox" placeholder="Enter Email"><br><br>
  <input type="button" value="Submit" id="btnSubmit">

  <script src="./selector.js"></script>
</body>
```

```
</html>
```

[selector.js](#)

```
$(document).ready(function () {  
    $("p").css("color", "green");  
  
    $(".inputBox").css({  
        "border": "2px solid blue",  
        "padding": "5px"  
    });  
  
    $("input[type='button']").click(function () {  
        alert("Form Submitted!");  
    });  
  
});
```

Actions :

A jQuery action is a built-in function that performs a task on the selected elements. After jQuery selects something, an action is used to modify, style, hide, insert, or animate it.

Action	Description
.text()	Get or set text content
.html()	Get or set HTML content
.val()	Read or update form field value
.addClass()	Add a CSS class
.removeClass()	Remove a CSS class
.toggleClass()	Add/remove class automatically
.css()	Apply or get CSS styles
.attr()	Get or set an attribute
.hide()	Hides the selected element
.fadeIn()	Fade the element into view

Example :

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Demo</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <style>
    .highlight {
      background: yellow;
      padding: 10px;
    }
    #box {
      width: 200px;
      height: 120px;
      background: lightblue;
      display: none;
    }
  </style>
</head>
<body>

<button id="showBox">Show Box</button>
<button id="toggleText">Toggle Text</button>
<button id="highlightText">Highlight</button>

<div id="box"></div>
<p id="text">This is sample text.</p>

<script src="./action.js"></script>
</body>
</html>
```

[action.js](#)

```
$(document).ready(function() {
  $("#showBox").click(function() {
    $("#box").fadeIn();
  });

  $("#toggleText").click(function() {
    $("#text").slideToggle();
  });
});
```

```
$("#highlightText").click(function() {  
    $("#text").addClass("highlight");  
});  
});
```

JQuery DOM manipulation

jQuery DOM manipulation refers to using the jQuery library to easily select, traverse, and modify the elements, attributes, and content of the Document Object Model (DOM) of a web page. It significantly simplifies the process compared to native JavaScript DOM methods.

jQuery provides a wide array of methods, which can be broadly grouped into the following categories:

1. Content Manipulation (Get/Set)

These functions are used to either access the current content of selected elements or update their text or HTML with new content.

Method	Description
.text()	Retrieves the plain text of the selected elements or updates their text content (any HTML tags inside are not kept).
.html()	Gets the HTML markup within the selected elements or replaces it with new HTML (supports tags).
.val()	Reads the value from form controls or assigns a new value to inputs, selects, and textareas.

Example :

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Methods Demo (External JS)</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script src="./contentManipulation.js"></script>
</head>

<body>
  <h2>jQuery Example - .text(), .html(), .val()</h2>
  <button id="showText">Show Text Using .text()</button>
  <p id="resultText"></p>

  <hr>
  <button id="showHtml">Show HTML Using .html()</button>
  <p id="resultHtml"></p>

  <hr>
  <input type="text" id="userInput" placeholder="Enter something...">
  <button id="showValue">Get Value Using .val()</button>
  <p id="resultVal"></p>

</body>
</html>
```

```
$(document).ready(function () {

  $("#showText").click(function () {
    $("#resultText").text("This message is displayed using the .text() method.");
  });

  $("#showHtml").click(function () {
```

```

        $("#resultHtml").html("<i>Formatted HTML content shown using</i>
<b>.html()</b>");
    });

    $("#showValue").click(function () {
        let value = $("#userInput").val();
        $("#resultVal").text("Entered value: " + value);
    });

});

```

2. Structure Manipulation (Adding/Removing Elements)

These functions help you insert new elements into the page or delete elements that are already present.

Method	Description
.append()	Adds new content to the end of the selected elements.
.prepend()	Adds new content to the start of the selected elements.
.after()	Inserts new content immediately after the selected elements as a sibling.
.before()	Inserts new content just before the selected elements as a sibling.
.remove()	Deletes the selected elements along with everything inside them.
.empty()	Clears all child elements/content but keeps the original element in place.

Example :

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>jQuery DOM Methods Example</title>
    <link rel="stylesheet" href="./StructureManipulation.css">
</head>
<body>

```



```
<h2>Student List Manager</h2>

<div id="box">
  <ul id="studentList">
    <li>Aditya</li>
    <li>Deepak</li>
    <li>Riya</li>
  </ul>
</div>

<button id="btnPrepend">Add Student at Start</button>
<button id="btnAppend">Add Student at End</button>
<button id="btnBefore">Add Message Before List</button>
<button id="btnAfter">Add Message After List</button>
<button id="btnEmpty">Empty List Items</button>
<button id="btnRemove">Remove Whole List</button>

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<script src="./StructureManipulation.js"></script>
</body>
</html>
```

StructureManipulation.css

```
body {
  font-family: Arial, sans-serif;
  padding: 20px;
}

#box {
  border: 2px solid black;
  padding: 15px;
  width: 300px;
  margin-bottom: 20px;
}

button {
  margin: 5px;
  padding: 8px 15px;
```

```
    cursor: pointer;
    font-size: 14px;
}
```

StructureManipulation.js

```
$("#btnPrepend").click(function () {
    $("#studentList").prepend("<li> New Student (Start)</li>");
});

$("#btnAppend").click(function () {
    $("#studentList").append("<li> New Student (End)</li>");
});

$("#btnBefore").click(function () {
    $("#box").before("<p style='color:blue;'> This is added BEFORE the
list.</p>");
});

$("#btnAfter").click(function () {
    $("#box").after("<p style='color:green;'> This is added AFTER the
list.</p>");
});

$("#btnEmpty").click(function () {
    $("#studentList").empty();
});

$("#btnRemove").click(function () {
    $("#box").remove();
});
```

3. Attribute and CSS Manipulation

These methods are used to change or retrieve attributes and styles.

Method	Description
.attr()	Retrieves the value of an attribute or assigns a new value (e.g., src, href, id).
.removeAttr()	Deletes a specified attribute from the selected elements.
.addClass()	Applies one or more CSS classes to the selected elements.
.removeClass()	Removes one or more CSS classes from the selected elements.
.toggleClass()	Switches a CSS class on or off depending on whether it is already present.
.css()	Gets the current value of a CSS property or updates the property with a new value.

Jquery Events / Event Handling :

JQuery events are fundamental to creating dynamic and interactive web pages. They represent actions that occur on the web page, and jQuery provides methods to handle these events by executing a function when the event occurs.

Event handling in jQuery refers to the process of responding to user interactions or browser actions on a webpage.

The general syntax for attaching an event handler is:

\$(selector).event_method(function);

1. Mouse Events

These are triggered by actions involving the mouse.

Event Method	Description	Example
.click()	Fires when an element is clicked.	<code>\$("#btn").click(function() { ... });</code>
.dblclick()	Fires when an element is double-clicked.	<code>\$("#p").dblclick(function() { ... });</code>

.mouseenter()	Fires when the mouse pointer enters the selected element.	<code>\$(".box").mouseenter(function() { ... });</code>
.mouseleave()	Fires when the mouse pointer leaves the selected element.	<code>\$(".box").mouseleave(function() { ... });</code>
.hover()	A shorthand for both <code>mouseenter</code> (first function) and <code>mouseleave</code> (second function).	<code>\$("li").hover(fn_in, fn_out);</code>

2. Keyboard Events

These are triggered by user interaction with the keyboard.

Event Method	Description	Example
.keydown()	Fires when a key is pressed down.	<code>\$(document).keydown(function() { ... });</code>
.keyup()	Fires when a key is released.	<code>\$("#inputField").keyup(function() { ... });</code>
.keypress()	Fires when a key is pressed and released (excluding system keys like Alt, Ctrl, Shift).	<code>\$("#text").keypress(function() { ... });</code>

3. Form Events

These are triggered by actions within form elements.

Event Method	Description	Example
.submit()	Fires when a form is submitted.	<code>\$("form").submit(function() { ... });</code>
.change()	Fires when the value of an input element (like <code><input></code> , <code><select></code> , or <code><textarea></code>) changes.	<code>\$("#selectList").change(function() { ... });</code>
.focus()	Fires when an element gets focus (e.g., clicking inside a text input).	<code>\$("#name").focus(function() { ... });</code>
.blur()	Fires when an element loses focus.	<code>\$("#name").blur(function() { ... });</code>

4. Document/Window Events

These are triggered by the browser window or the document itself.

Event Method	Description	Example
.ready()	Executes a function after the DOM is fully loaded. (Most common)	<code>\$(document).ready(function() { ... });</code>

.load()	Executes a function when an element (like an image) or the entire page is fully loaded.	<code>\$(window).load(function() { ... });</code>
.resize()	Executes a function when the browser window is resized.	<code>\$(window).resize(function() { ... });</code>
.scroll()	Executes a function when the user scrolls the designated element or the document.	<code>\$(window).scroll(function() { ... });</code>

Example :

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>

<body>
  <h2 id="box">Hover or Click Me</h2>
  <input type="text" id="keyInput" placeholder="Press any key">

  <form id="myForm">
    <input type="text" id="formInput" placeholder="Enter name">
    <button type="submit">Submit</button>
  </form>
  <p id="result"></p>
  <script src="./event.js"></script>
</body>
</html>
```

[event.js](#)

```
$(document).ready(function () {  
    $("#box").click(function () {  
        $("#result").text("Mouse Click Detected");  
    });  
  
    $("#box").mouseenter(function () {  
        $("#result").text("Mouse Enter Event");  
    });  
  
    $("#box").mouseleave(function () {  
        $("#result").text("Mouse Leave Event");  
    });  
  
    $("#keyInput").keydown(function () {  
        $("#result").text("Key Down Event");  
    });  
  
    $("#keyInput").keyup(function () {  
        $("#result").text("Key Up Event");  
    });  
  
    $("#keyInput").keypress(function () {  
        $("#result").text("Key Press Event");  
    });  
  
    $("#myForm").submit(function (e) {  
        e.preventDefault();  
        $("#result").text("Form Submitted: " + $("#formInput").val());  
    });  
});
```

jQuery Effects

jQuery Effects are built-in features that allow you to create smooth visual transitions on web elements. They help you show, hide, fade, slide, or animate elements without writing complex code. These effects make webpages feel more interactive and lively.

1. Show/Hide Effects

This group of effects controls the visibility of elements. You can display hidden items, hide visible ones, or switch their visibility with a single method.

Method	Description
show()	Makes an element appear on the page.
hide()	Makes an element disappear from the page.
toggle()	Alternates an element between visible and hidden states.

2. Fade Effects

Fade effects smoothly change an element's opacity, creating a gradual appearance or disappearance.

Method	Description
fadeIn()	Gradually increases the opacity to make an element visible.
fadeOut()	Gradually decreases the opacity to hide the element.
fadeToggle()	Switches between fading in and fading out.
fadeTo()	Fades an element to a specific opacity level you choose.

3. Slide Effects

Slide effects reveal or hide content with a vertical sliding motion, similar to opening or closing a drawer.

Method	Description
slideDown()	Reveals the element by sliding it downward.
slideUp()	Hides the element by sliding it upward.
slideToggle()	Alternates between sliding down and sliding up.

4. Animate Effects

The `animate()` method lets you create custom animations by gradually changing CSS properties like width, height, margin, opacity, etc.

Method	Description
animate()	Allows you to design your own animation by modifying CSS properties step-by-step.

Example :

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <style>
    #panel {
      width: 200px;
      height: 150px;
      background: pink;
      margin: 20px;
    }
  </style>
</head>

<body>
  <h2>jQuery Effects Demo</h2>
  <button id="btnShow">Show Box</button>
  <button id="btnHide">Hide Box</button>
  <button id="btnToggle">Toggle Box</button>

  <button id="btnFadeIn">Fade In</button>
  <button id="btnFadeOut">Fade Out</button>
  <button id="btnFadeToggle">Fade Toggle</button>
  <button id="btnFadeTo">Fade to 50%</button>
```

```
<button id="btnSlideDown">Slide Down</button>
<button id="btnSlideUp">Slide Up</button>
<button id="btnSlideToggle">Slide Toggle</button>

<button id="btnAnimate">Animate Box</button>

<div id="panel"></div>

<script src="./Effects.js"></script>
</body>
</html>
```

[Effects.js](#)

```
$(document).ready(function () {
    $("#btnShow").click(function () {
        $("#panel").show();
    });

    $("#btnHide").click(function () {
        $("#panel").hide();
    });

    $("#btnToggle").click(function () {
        $("#panel").toggle();
    });

    $("#btnFadeIn").click(function () {
        $("#panel").fadeIn();
    });

    $("#btnFadeOut").click(function () {
        $("#panel").fadeOut();
    });

    $("#btnFadeToggle").click(function () {
        $("#panel").fadeToggle();
    });

    $("#btnFadeTo").click(function () {
        $("#panel").fadeTo("slow", 0.5);
    });
});
```

```

});

$("#btnSlideDown").click(function () {
    $("#panel").slideDown();
});

$("#btnSlideUp").click(function () {
    $("#panel").slideUp();
});

$("#btnSlideToggle").click(function () {
    $("#panel").slideToggle();
});

$("#btnAnimate").click(function () {
    $("#panel").animate({
        width: "300px",
        height: "100px",
        opacity: 0.4,
        borderRadius: "20px"
    }, 1000);
});
});

```

jQuery Traversing

Query Traversing means moving through the DOM tree to locate elements in relation to another element.

It helps you reach parents, children, siblings, or filter matched elements easily.

Category	Method	Description
Ancestors	parent()	Selects the immediate parent element
	parents()	Selects all ancestor elements
	parentsUntil()	Selects ancestors up to a specified element
Descendants	children()	Selects only direct children

	find()	Selects all matching descendant elements
Siblings	siblings()	Selects all siblings of the element
	next()	Selects the next sibling
	prev()	Selects the previous sibling
	nextAll()	Selects all following siblings
	prevAll()	Selects all previous siblings
	nextUntil()	Selects all following siblings until a specific element
	prevUntil()	Selects all previous siblings until a specific element
Filtering	eq()	Selects element at specific index
	first()	Selects the first element
	last()	Selects the last element
	filter()	Selects elements that match a condition
	not()	Excludes elements that match a condition

Example :

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Traversing Demo</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <style>
    ul, div {
      margin-bottom: 20px;
    }
    .highlight {
      background-color: yellow;
    }
  </style>
</head>
<body>

<h2>Ancestors Example</h2>
<div id="ancestorDiv">
```

```

    <ul>
      <li><span id="item">Click Me</span></li>
    </ul>
  </div>

<h2>Descendants Example</h2>
<div id="container">
  <p>Paragraph 1</p>
  <p class="special">Paragraph 2</p>
  <p>Paragraph 3</p>
</div>

<h2>Siblings Example</h2>
<ul>
  <li>One</li>
  <li id="mid">Two</li>
  <li>Three</li>
  <li>Four</li>
</ul>

<h2>Filtering Example</h2>
<p>Para 0</p>
<p>Para 1</p>
<p class="highlight">Para 2</p>
<p>Para 3</p>

<script src="./traversing.js"></script>

</body>
</html>

```

[traversing.js](#)

```

$(document).ready(function() {

  // Ancestors
  $("#item").click(function() {
    $("#item").parent().css("color", "red");
    $("#item").parents().css("border", "1px solid blue");
  });

  // Descendants
  $("#container").children().css("color", "green");

```

```

$("#container").find(".special").css("font-weight", "bold");

// Siblings
$("#mid").siblings().css("color", "blue");
$("#mid").next().css("font-size", "20px");
$("#mid").prev().css("font-weight", "bold");

// Filtering
$("p").eq(1).css("color", "red");
$("p").first().css("background", "yellow");
$("p").filter(".highlight").css("font-weight", "bold");
$("p").not(".highlight").css("opacity", "0.6");

});

```

Handling different versions of jQuery :

Including two or more jQuery versions on the same webpage can cause conflicts, because both versions try to use the same \$ and jQuery identifiers. The last loaded version usually overwrites the previous one, which can break scripts written for older versions.

To prevent conflicts, there are two main strategies:

1. Using noConflict()

- Releases the \$ and jQuery identifiers for a specific jQuery version.
- Assign the version to a custom variable.
- Allows multiple versions to coexist on the same page.

Syntax :

```
var jq1 = jQuery.noConflict(true);
```

Explanation :

true ensures both \$ and jQuery are released.

jq1 becomes the reference for this jQuery version.

2. Load Versions in Proper Order

- Always load older versions first, newer versions later.
- This ensures code written for older versions still works.

```
<!DOCTYPE html>
<html>
<head>
  <title>Multiple jQuery Versions Example</title>

  <!-- Load jQuery 1.x first -->
  <script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
  <script>
    var jq1 = jQuery.noConflict(true);
  </script>

  <!-- Load jQuery 3.x next -->
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script>
    var jq3 = jQuery.noConflict(true);
  </script>

  <style>
    #box1, #box2 {
      width: 150px;
      height: 150px;
      margin: 10px;
      display: inline-block;
    }
    #box1 { background-color: lightblue; }
    #box2 { background-color: lightgreen; }
  </style>
</head>
<body>

<h2>Multiple jQuery Versions Demo</h2>

<div id="box1">Box 1 (jQuery 1.x)</div>
<div id="box2">Box 2 (jQuery 3.x)</div>
<button id="btn1">Change Box 1</button>
```

```
<button id="btn2">Change Box 2</button>
```

```
<script src="./conflict.js"></script>  
</body>  
</html>
```

```
// Using jQuery 1.x
```

```
jq1(document).ready(function() {  
    jq1("#btn1").click(function() {  
        jq1("#box1").text("Changed by jQuery 1.x").css({  
            "background-color": "orange",  
            "color": "white"  
        });  
    });  
});
```

```
// Using jQuery 3.x
```

```
jq3(document).ready(function() {  
    jq3("#btn2").click(function() {  
        jq3("#box2").text("Changed by jQuery 3.x").css({  
            "background-color": "purple",  
            "color": "yellow"  
        });  
    });  
});
```